

The Critical Line Algorithm

A parametric active-set method for the entire mean–variance efficient frontier, as implemented in `cvxcla`

Thomas Schmelzer
Jebel Quant Research, Abu Dhabi

June 16, 2026

Abstract

We describe the Critical Line Algorithm (CLA) of Markowitz as implemented in the `cvxcla` package. The CLA computes the *entire* mean–variance efficient frontier of a long-only (or box-constrained) portfolio problem exactly, rather than sampling it point by point. It exploits the fact that the efficient frontier is piecewise linear in the weight space: between two consecutive *turning points* the optimal weights are an affine function $w(\lambda) = \alpha + \lambda\beta$ of a single scalar parameter λ that trades expected return against variance. The algorithm starts at the maximum-return portfolio ($\lambda = \infty$) and walks the frontier down to the minimum-variance portfolio ($\lambda = 0$), changing the status of exactly one asset at each turning point. Each step amounts to a single solve of a reduced Karush–Kuhn–Tucker (KKT) system, handled here by block elimination through a small Schur complement. We document the problem formulation, the optimality conditions, the event logic that detects turning points, the anti-cycling rule used on degenerate problems, and the covariance-operator abstraction that lets structured risk models (diagonal plus low rank) avoid ever materialising a dense $n \times n$ matrix.

Contents

1	Introduction	2
2	Problem formulation	2
2.1	The parametric quadratic program	2
2.2	Optimality conditions	3
2.3	Piecewise-linearity: the affine segment	3
3	The first turning point	3
4	The turning-point iteration	4
4.1	Solving the reduced KKT system by block elimination	4
4.2	Reduced gradients for blocked assets	4
4.3	Event detection	5
4.4	Anti-cycling on degenerate problems	5
4.5	Termination and the minimum-variance endpoint	6
4.6	The complete procedure	6

5	The covariance operator abstraction	6
5.1	Dense covariance	6
5.2	Factor (diagonal-plus-low-rank) covariance	7
6	The efficient frontier object	7
7	Complexity and properties	7
8	Numerical experiment	8
8.1	Illustration: a 20-asset frontier	8
8.2	Runtime versus problem size	8
9	Conclusion	9
	Acknowledgements	9

1 Introduction

The mean–variance portfolio selection problem of [Markowitz \(1952\)](#) asks for the allocation of capital across n assets that minimises risk for a given level of expected return. Sweeping the return level produces a one-parameter family of optimal portfolios whose risk–return image is the *efficient frontier*. The Critical Line Algorithm (CLA), introduced by [Markowitz \(1956\)](#) and developed in [Markowitz \(1959\)](#), computes this frontier *exactly and in full*: it returns a finite list of *turning points* between which the frontier is described in closed form, so no discretisation or repeated re-optimisation is required. It belongs to a broad family of parametric quadratic-programming and active-set methods for the efficient frontier ([Wolfe, 1959](#); [Perold, 1984](#); [Niedermayer and Niedermayer, 2010](#); [Bailey and López de Prado, 2013](#)); [Markowitz and Todd \(2000\)](#) give the canonical description and a reference implementation, while [Perold \(1984\)](#), [Stein et al. \(2008\)](#) and [Hirschberger et al. \(2010\)](#) develop large-scale and active-set variants.

This document describes the implementation in the `cvxcla` package. The core of the implementation is the class `CLA` in `src/cvxcla/cla.py`; it is supported by `first.py` (the starting portfolio), `operators.py` (the covariance backend), and `types.py` (the turning-point and frontier data structures). The notation below follows the code: μ is the vector of expected returns, Σ the covariance, A, b the linear equality constraints, and ℓ, u the box bounds on the weights.

2 Problem formulation

2.1 The parametric quadratic program

Let $w \in \mathbb{R}^n$ be the vector of portfolio weights, $\mu \in \mathbb{R}^n$ the expected returns, $\Sigma \in \mathbb{R}^{n \times n}$ the (symmetric, positive semidefinite) covariance matrix, $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$ the equality constraints, and $\ell, u \in \mathbb{R}^n$ the lower and upper bounds. The implementation traces the frontier through the *return-parametrised* quadratic program

$$\begin{aligned}
& \min_{w \in \mathbb{R}^n} && \frac{1}{2} w^\top \Sigma w - \lambda \mu^\top w \\
& \text{subject to} && Aw = b, \\
& && \ell \leq w \leq u,
\end{aligned} \tag{1}$$

where $\lambda \geq 0$ is a scalar that weights expected return against variance. The canonical instance has a single equality constraint, the budget constraint $\mathbf{1}^\top w = 1$ (so $A = \mathbf{1}^\top$ and $b = 1$), but (1) and the implementation allow any m linear equalities.

As λ decreases from $+\infty$ to 0, the optimiser of (1) moves from the maximum expected-return portfolio (subject to the constraints) to the global minimum-variance portfolio. The locus of these optimisers is exactly the efficient frontier.

2.2 Optimality conditions

At a fixed λ the program (1) is convex, so the Karush–Kuhn–Tucker (KKT) conditions are necessary and sufficient. Partition the assets into a *free* set $\mathcal{F}$ (weights strictly inside their bounds) and a *blocked* set $\mathcal{B} = \mathcal{F}^c$ (weights pinned at a bound, $w_i \in \{\ell_i, u_i\}$). For the free assets the stationarity condition holds with no bound multiplier:

$$\Sigma_{\mathcal{F}\mathcal{F}} w_{\mathcal{F}} + \Sigma_{\mathcal{F}\mathcal{B}} w_{\mathcal{B}} + A_{\mathcal{F}}^\top \nu = \lambda \mu_{\mathcal{F}}, \quad A_{\mathcal{F}} w_{\mathcal{F}} + A_{\mathcal{B}} w_{\mathcal{B}} = b, \quad (2)$$

where $\nu \in \mathbb{R}^m$ is the multiplier of the equality constraints and the subscripts select the free/blocked rows and columns. The blocked weights $w_{\mathcal{B}}$ are known (they sit on ℓ or u), so (2) is a linear system in the unknowns $(w_{\mathcal{F}}, \nu)$.

2.3 Piecewise-linearity: the affine segment

The crucial structural fact is that, *while the free/blocked partition is fixed*, the right-hand side of (2) is affine in λ and the system matrix is constant. Hence the solution is affine in λ :

$$w(\lambda) = \alpha + \lambda \beta, \quad (3)$$

where $\alpha, \beta \in \mathbb{R}^n$ are constant on the segment (the blocked entries of β are zero; the blocked entries of α equal the active bound values). The implementation calls these vectors `r_alpha` and `r_beta`. Equation (3) is the *critical line*: a straight line in weight space valid for an interval of λ . A *turning point* is a value of λ at which the partition $(\mathcal{F}, \mathcal{B})$ must change, i.e. an endpoint of such an interval.

3 The first turning point

The walk starts at $\lambda = +\infty$, where variance is irrelevant and (1) degenerates into “earn the most expected return that the bounds and budget allow.” This portfolio is computed in closed form by `init_algo` (`src/cvxcla/first.py`):

1. Initialise every weight at its lower bound, $w \leftarrow \ell$.
2. Sort the assets by descending expected return.
3. Walk down that order, raising each asset’s weight from ℓ_i towards u_i , accumulating capital, until the budget $\sum_i w_i = 1$ is reached (within a floating-point tolerance). The last asset touched is only *partially* filled — it absorbs the residual $1 - \sum_i w_i$ — and is the single *free* asset of the first turning point. All others are blocked, either at their lower bound (not yet reached) or upper bound (fully filled).

The result is “the smallest subset of assets with highest return whose upper bounds sum to at least one,” which is the maximum-return vertex of the feasible polytope. It is returned as a

TurningPoint with $\lambda = \infty$, a weight vector, and a boolean **free** mask. The tolerance $1 - 10^{-12}$ on the budget test matters: the increment $1 - \sum_i w_i$ can only reach 1 up to rounding, and without the slack the loop would skip past the genuinely interior asset and mark the next (zero-weight) asset as free.

4 The turning-point iteration

From the first turning point the algorithm repeatedly: (i) solves the reduced KKT system for the current partition to obtain the affine segment (3); (ii) finds the largest λ below the current one at which an *event* occurs; (iii) updates the partition by the single asset responsible for that event; and (iv) records the new turning point. It stops when λ reaches 0.

4.1 Solving the reduced KKT system by block elimination

For the free partition $\mathcal{F}$ the system (2) is the saddle-point (KKT) system

$$\begin{bmatrix} \Sigma_{\mathcal{F}\mathcal{F}} & A_{\mathcal{F}}^T \\ A_{\mathcal{F}} & 0 \end{bmatrix} \begin{bmatrix} w_{\mathcal{F}} \\ \nu \end{bmatrix} = \begin{bmatrix} r_1 \\ r_2 \end{bmatrix}. \quad (4)$$

Because the right-hand side splits into a constant part and a λ -part, the implementation solves (4) for *two* right-hand sides simultaneously (the “ α system” and the “ β system”):

$$r_1^\alpha = -\Sigma_{\mathcal{F}\mathcal{B}} w_{\mathcal{B}}, \quad r_2^\alpha = b - A_{\mathcal{B}} w_{\mathcal{B}}; \quad r_1^\beta = \mu_{\mathcal{F}}, \quad r_2^\beta = 0.$$

Rather than assembling and factoring the full saddle matrix, the code eliminates the free block first (*block elimination* / Schur complement). With

$$Y = \Sigma_{\mathcal{F}\mathcal{F}}^{-1} A_{\mathcal{F}}^T, \quad z^\alpha = \Sigma_{\mathcal{F}\mathcal{F}}^{-1} r_1^\alpha, \quad z^\beta = \Sigma_{\mathcal{F}\mathcal{F}}^{-1} r_1^\beta,$$

obtained from a *single* multi-column solve against $\Sigma_{\mathcal{F}\mathcal{F}}$, the equality multipliers come from the $m \times m$ Schur complement $S = A_{\mathcal{F}} \Sigma_{\mathcal{F}\mathcal{F}}^{-1} A_{\mathcal{F}}^T = A_{\mathcal{F}} Y$:

$$\nu^\alpha = S^{-1} (A_{\mathcal{F}} z^\alpha - r_2^\alpha), \quad \nu^\beta = S^{-1} (A_{\mathcal{F}} z^\beta). \quad (5)$$

Back-substitution then gives the free part of the affine segment,

$$\alpha_{\mathcal{F}} = z^\alpha - Y \nu^\alpha, \quad \beta_{\mathcal{F}} = z^\beta - Y \nu^\beta, \quad (6)$$

while the blocked entries are fixed: $\alpha_{\mathcal{B}} = w_{\mathcal{B}}$ and $\beta_{\mathcal{B}} = 0$. The dominant cost is the multi-right-hand-side solve against the free block $\Sigma_{\mathcal{F}\mathcal{F}}$; the Schur solve (5) is only $m \times m$, and for the standard budget constraint $m = 1$, so it is a scalar division.

4.2 Reduced gradients for blocked assets

To decide when a blocked asset should re-enter the free set we need the reduced gradient of the Lagrangian at the blocked coordinates. The implementation forms two vectors, again split into a constant and a λ -linear part:

$$\gamma = \Sigma \alpha + A^T \nu^\alpha, \quad (7)$$

$$\delta = \Sigma \beta + A^T \nu^\beta - \mu. \quad (8)$$

For a blocked asset i the quantity $\gamma_i + \lambda\delta_i$ is (proportional to) its bound multiplier along the segment. While that multiplier keeps the correct sign the asset is correctly blocked; the λ at which it would change sign is exactly the value at which the asset wants to become free. (The covariance products $\Sigma\alpha$ and $\Sigma\beta$ are applied through the covariance operator of §5, never as an explicit dense multiply when a structured backend is in use.)

4.3 Event detection

Walking down from the current λ , the segment (3) stays optimal until one of two things happens. The implementation enumerates four event *types*, two for each direction:

A free asset reaches a bound (free $\rightarrow$ blocked). A free weight evolves as $w_i(\lambda) = \alpha_i + \lambda\beta_i$. It meets a bound at

$$\lambda_i = \begin{cases} \frac{u_i - \alpha_i}{\beta_i}, & \beta_i < 0 \quad (\text{heads to the } \textit{upper} \text{ bound}), \\ \frac{\ell_i - \alpha_i}{\beta_i}, & \beta_i > 0 \quad (\text{heads to the } \textit{lower} \text{ bound}). \end{cases} \quad (9)$$

A blocked asset’s multiplier changes sign (blocked $\rightarrow$ free). A blocked asset wants to become free at

$$\lambda_i = -\frac{\gamma_i}{\delta_i}, \quad (10)$$

admissible only when the asset is at its *upper* bound with $\delta_i < 0$, or at its *lower* bound with $\delta_i > 0$.

These candidate ratios are assembled into a matrix $L \in \mathbb{R}^{n \times 4}$ (one column per event type), with entries set to $-\infty$ where the event cannot occur. Two numerical guards apply:

- **Slope floor.** Slopes β_i and derivatives δ_i are tested against $\sqrt{\varepsilon_{\text{mach}}}$, not the user tolerance. A free weight with a *tiny but nonzero* slope still crosses a bound given a long enough λ range, so filtering at the coarser tolerance would let weights drift out of bounds; only slopes at floating-point noise level (below $\sqrt{\varepsilon_{\text{mach}}}$) are discarded, since the enormous ratios they would produce merely amplify rounding error.
- **λ window.** The segment is valid only for λ at or below the current value, because the frontier is traced with non-increasing λ . Candidate ratios above the current λ (by more than the tolerance) are spurious crossings outside the segment and are set to $-\infty$.

The next turning point is at $\lambda^* = \max_{i,t} L_{it}$. If $\lambda^* < 0$ the frontier is complete.

4.4 Anti-cycling on degenerate problems

On degenerate problems — tied expected returns, duplicated assets, or several constraints active at once — multiple events can occur at the same ratio λ^* . A naive “pick any maximiser” rule can cycle (the same partitions recurring without progress). The implementation uses a **Bland-style rule**: among all events within the tolerance of λ^* it selects the one with the lowest asset index, and within an asset the lowest event-type column. This makes the choice deterministic and guarantees termination, resolving degeneracies one asset per iteration.

The asset and event type selected determine the partition update: event types “free $\rightarrow$ blocked” clear the asset’s **free** flag, while “blocked $\rightarrow$ free” set it. Exactly one asset changes status. The new weight vector $w(\lambda^*) = \alpha + \lambda^*\beta$ is stored as the next turning point.

4.5 Termination and the minimum-variance endpoint

The loop runs while $\lambda > 0$. When no admissible event has a positive ratio the walk has reached the global minimum-variance portfolio; the algorithm appends a final turning point at $\lambda = 0$ with weights α (the constant part of the current segment). A hard iteration cap of $100(n + 1)$ guards against non-termination: since each step fixes the status of at least one asset, a correct trace runs $O(n)$ times, and far exceeding this signals a failure (e.g. cycling), which is raised loudly rather than allowed to hang. Every stored turning point is validated to satisfy the bounds and the budget constraint before being accepted.

4.6 The complete procedure

Algorithm 1 collects the steps.

Algorithm 1. Critical Line Algorithm (as implemented in `cvxcla`).

Input: mean μ , covariance Σ , bounds ℓ, u , equality constraints A, b , tolerance τ .

1. Compute $w^{(0)} \leftarrow \text{init_algo}(\mu, \ell, u)$, the maximum-return vertex (§3); append it with $\lambda \leftarrow \infty$.
2. **While** $\lambda > 0$:
 - (a) Read the free/blocked partition $(\mathcal{F}, \mathcal{B})$ of the last turning point; classify $\mathcal{B}$ into *at-upper/at-lower* and fix $w_{\mathcal{B}}$ to the active bounds.
 - (b) Solve the free block $\Sigma_{\mathcal{F}\mathcal{F}}$ for Y, z^{α}, z^{β} (multi-RHS, §4.1).
 - (c) Form the Schur complement $S = A_{\mathcal{F}}Y$ and solve (5) for $\nu^{\alpha}, \nu^{\beta}$; back-substitute (6) to get α, β .
 - (d) Compute the reduced gradients γ, δ via (7)–(8).
 - (e) Assemble the candidate ratios L from (9)–(10); apply the slope floor and the λ window. Set $\lambda^* \leftarrow \max L$; **if** $\lambda^* < 0$ **break**.
 - (f) Pick the event by the Bland-style rule (§4.4) and flip the **free** status of that one asset. Set $\lambda \leftarrow \lambda^*$ and append the turning point $w \leftarrow \alpha + \lambda^* \beta$.
3. Append the final turning point $w \leftarrow \alpha$ at $\lambda = 0$ (the minimum-variance portfolio) and **return** the list of turning points.

5 The covariance operator abstraction

The turning-point loop touches the covariance matrix through only three operations: a full matrix–vector product Σx (`matvec`); a solve against the free block, $\Sigma_{\mathcal{F}\mathcal{F}}^{-1}R$ for a multi-column right-hand side R (`solve_free`); and a free-to-blocked cross product $\Sigma_{\mathcal{F}\mathcal{B}}x_{\mathcal{B}}$ (`cross`). The package captures exactly this contract in the `CovarianceOperator` protocol (`src/cvxcla/operators.py`), so the algorithm never assumes an explicit matrix. Two backends implement it.

5.1 Dense covariance

`DenseCovariance` wraps an explicit symmetric $n \times n$ matrix and implements the three operations with plain `numpy` calls: `matvec` is Σx , `solve_free` is `np.linalg.solve` on the principal submatrix

$\Sigma_{\mathcal{FF}}$, and `cross` is the off-diagonal block product. This reproduces exactly the behaviour of the classical CLA on a raw matrix and is the reference backend.

5.2 Factor (diagonal-plus-low-rank) covariance

`FactorCovariance` represents a structured risk model

$$\Sigma = \text{diag}(d) + U \Delta U^\top, \quad d \in \mathbb{R}_{>0}^n, U \in \mathbb{R}^{n \times k}, \Delta \in \mathbb{R}^{k \times k}, \quad (11)$$

the form taken by factor models (Sharpe, 1963) and by random-matrix-theory-cleaned covariances (constant-residual-eigenvalue / eigenvalue clipping). The free-block solve never forms $\Sigma_{\mathcal{FF}}$; instead it uses the Woodbury identity

$$\Sigma_{\mathcal{FF}}^{-1} = D_{\mathcal{F}}^{-1} - D_{\mathcal{F}}^{-1} U_{\mathcal{F}} W^{-1} U_{\mathcal{F}}^\top D_{\mathcal{F}}^{-1}, \quad W = \Delta^{-1} + U_{\mathcal{F}}^\top D_{\mathcal{F}}^{-1} U_{\mathcal{F}} \in \mathbb{R}^{k \times k}, \quad (12)$$

so a solve costs $O(n_{\mathcal{F}} k^2 + k^3)$ rather than $O(n_{\mathcal{F}}^3)$, and no $n \times n$ matrix is ever allocated — memory and per-solve cost scale as $O(nk)$ instead of $O(n^2)$. The cross product (11) contributes only through its low-rank part, since the diagonal has no off-diagonal block: $\Sigma_{\mathcal{FB}} x_{\mathcal{B}} = U_{\mathcal{F}} \Delta (U_{\mathcal{B}}^\top x_{\mathcal{B}})$. Because the CLA accesses the covariance solely through the protocol, switching from a dense matrix to a factor model requires no change to the turning-point loop.

6 The efficient frontier object

The list of turning points is wrapped in a `Frontier` object (`src/cvxcla/types.py`). Each `FrontierPoint` carries a weight vector and exposes its expected return $\mu^\top w$ and variance $w^\top \Sigma w$. Between adjacent turning points the frontier in *weight* space is the straight segment (3), so the object can `interpolate` any number of intermediate portfolios by convex combination. Derived quantities — volatility, Sharpe ratio — are computed pointwise. The maximum-Sharpe portfolio is found by a one-dimensional search over the convex combination of the two turning points bracketing the discrete Sharpe maximum, exploiting the same piecewise-linear weight structure.

7 Complexity and properties

- **Exactness.** The frontier is returned exactly as a finite set of turning points; between them the closed-form segment (3) holds. No discretisation error is introduced.
- **Number of turning points.** Each iteration changes exactly one asset’s status, so the number of turning points is $O(n)$ in practice; the implementation caps iterations at $100(n + 1)$ as a safety net.
- **Per-iteration cost.** Dominated by the free-block solve: $O(n_{\mathcal{F}}^3)$ for the dense backend, or $O(n_{\mathcal{F}} k^2 + k^3)$ for the factor backend via Woodbury (12). The Schur complement (5) adds only an $m \times m$ solve ($m = 1$ for the budget constraint).
- **Robustness on degenerate data.** The Bland-style tie-break (§4.4) and the $\sqrt{\varepsilon_{\text{mach}}}$ slope floor (§4.3) make the trace deterministic and prevent both cycling and out-of-bounds drift.

8 Numerical experiment

To illustrate the algorithm we first trace the entire efficient frontier of a small problem — small enough that its piecewise-linear corner structure is plainly visible — and then measure how the runtime grows with the number of assets. Both experiments are reproducible from fixed random seeds: `experiments/frontier_20x50.py` and `experiments/runtime_scaling.py`.

8.1 Illustration: a 20-asset frontier

Setup. We simulate $T = 50$ daily returns for $N = 20$ assets from a $K = 5$ factor model. As the risk model we use the factor covariance itself, $\Sigma = \text{diag}(d) + U \Delta U^\top$ (standard practice — a structured risk model rather than the noisy sample covariance), and we estimate the expected returns $\mu \in \mathbb{R}^{20}$ by the sample mean over the 50 days. We trace the long-only, fully-invested frontier, i.e. (1) with the budget constraint $\mathbf{1}^\top w = 1$ and box bounds $0 \leq w \leq 1$.

Results. The algorithm returns the *entire* frontier as a sequence of **21 turning points** (Figure 1); between consecutive points the frontier is the exact affine segment (3), so these 21 corner portfolios describe the whole efficient set with no discretisation. The piecewise-linear corners are clearly visible towards the low-variance end of the curve. Tracing this small problem takes on the order of a millisecond. Because the dense matrix and the structured `FactorCovariance` (Woodbury) backend represent the *same* Σ , the two backends return the identical 21-point frontier — the Woodbury identity is exact, so it changes only the cost of each solve, never the result; its runtime advantage as the universe grows is the subject of §8.2. (As an aside, replacing Σ by a low-rank *approximation* of the full sample covariance — its leading eigenpairs plus a diagonal residual — yields a near-identical frontier differing by at most one corner, which is why factor risk models are a practical substitute for the raw sample covariance.)

8.2 Runtime versus problem size

We next trace the full frontier for $n \in \{20, 40, 80, 160, 320, 640\}$ assets, each from a ground-truth $K = 10$ factor model so that the dense and factor backends again solve the identical Σ and return the same frontier. For reference we also time PyPortfolioOpt’s CLA (Bailey and López de Prado, 2013), the widely-used reference implementation of the critical-line algorithm, on the same dense problem. The number of turning points tracks n closely (21, 42, 81, 159, 322, 641 respectively, and PyPortfolioOpt finds essentially the same set), confirming the $O(n)$ count of §8.

Figure 2 plots the median trace time against n on log-log axes. A least-squares fit over the largest four sizes gives an empirical exponent of ≈ 2.2 for the `cvxcla` dense backend and ≈ 1.3 for the factor backend: the dense per-iteration solve scales like $O(n_{\mathcal{F}}^3)$ over $O(n)$ iterations, while the Woodbury solve with fixed K stays close to linear in n . The dense and factor backends are comparable for small universes (the Woodbury correction carries a fixed $K \times K$ overhead), but the factor backend pulls away as n grows — from rough parity at $n = 40$ to a $\approx 8\times$ speedup at $n = 640$ (Table 1). PyPortfolioOpt’s pure-Python implementation scales far more steeply ($\approx n^{3.4}$): at $n = 640$ it takes about 253 seconds, so `cvxcla`’s dense backend is roughly $330\times$ faster and the factor backend roughly $2800\times$ faster. Much of that gap is implementation (a vectorised block-elimination over `numpy` versus element-wise Python loops); the factor backend adds the asymptotic Woodbury advantage on top.

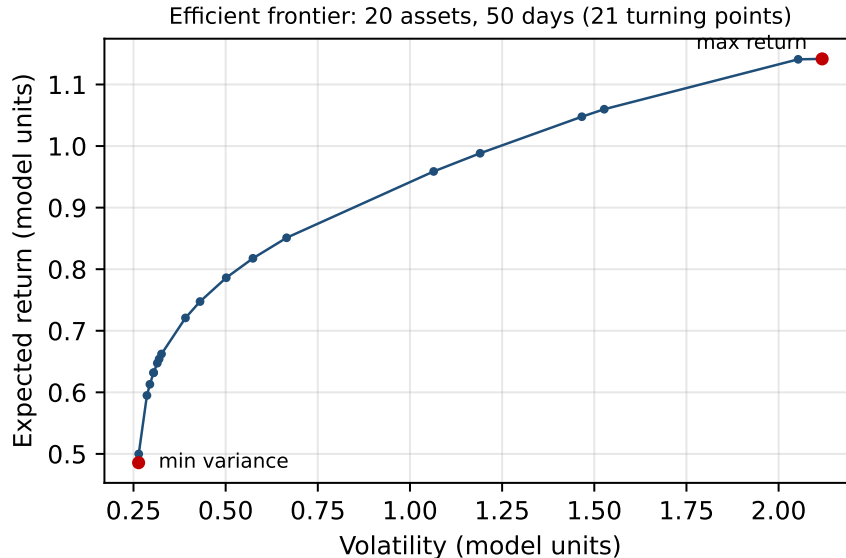


Figure 1: The efficient frontier of the 20-asset, 50-day problem in expected-return / volatility space, computed exactly by the Critical Line Algorithm. Each marker is one of the 21 turning points; the endpoints are the maximum-return portfolio ($\lambda = \infty$) and the global minimum-variance portfolio ($\lambda = 0$).

Remark on conditioning. These traces run cleanly because the expected returns are well separated and the covariance is well conditioned. On strongly degenerate inputs — (near-)tied expected returns, duplicated assets, or an ill-conditioned free block — the event logic can encounter many coincident ratios; the Bland-style rule of §4.4 resolves these deterministically, but very tie-heavy problems remain numerically delicate.

9 Conclusion

The Critical Line Algorithm computes the complete mean–variance efficient frontier by recognising its piecewise-linear structure in weight space and walking from the maximum-return portfolio to the minimum-variance portfolio, flipping one asset between the free and blocked sets at each turning point. The `cvxcla` implementation realises each step as a single block-eliminated KKT solve, hardens the event logic against degeneracy and floating-point noise, and abstracts the covariance behind a small operator protocol so that structured risk models obtain the same exact frontier at a fraction of the dense cost.

Acknowledgements

The author thanks Stephen Boyd and Philipp Schiele. The first versions of the solver were implemented while the author stayed for a sabbatical at Boyd’s group at Stanford University.

n	Turning points	PyPortfolioOpt [ms]	cvxcla dense [ms]	cvxcla factor [ms]
20	21	7.4	1.3	1.5
40	42	40.3	3.2	3.2
80	81	212.1	7.2	6.3
160	159	1,477	23.5	14.3
320	322	15,932	113.8	33.4
640	641	253,307	762.7	91.6

Table 1: Frontier trace time versus number of assets n ($K = 10$ factors; `cvxcla` times are the median of 3 repetitions, PyPortfolioOpt a single run). All implementations return the same $O(n)$ -point frontier. Empirically the times scale like $\approx n^{3.4}$ (PyPortfolioOpt), $\approx n^{2.2}$ (`cvxcla` dense) and $\approx n^{1.3}$ (`cvxcla` factor).

References

- H. M. Markowitz. Portfolio selection. *The Journal of Finance*, 7(1):77–91, 1952.
- H. M. Markowitz. The optimization of a quadratic function subject to linear constraints. *Naval Research Logistics Quarterly*, 3(1–2):111–133, 1956.
- H. M. Markowitz. *Portfolio Selection: Efficient Diversification of Investments*. John Wiley & Sons, New York, 1959.
- A. Niedermayer and D. Niedermayer. Applying Markowitz’s critical line algorithm. In *Handbook of Portfolio Construction*, pages 383–400. Springer, 2010.
- D. H. Bailey and M. López de Prado. An open-source implementation of the critical-line algorithm for portfolio optimization. *Algorithms*, 6(1):169–196, 2013.
- P. Wolfe. The simplex method for quadratic programming. *Econometrica*, 27(3):382–398, 1959.
- W. F. Sharpe. A simplified model for portfolio analysis. *Management Science*, 9(2):277–293, 1963.
- A. F. Perold. Large-scale portfolio optimization. *Management Science*, 30(10):1143–1160, 1984.
- H. M. Markowitz and G. P. Todd. *Mean-Variance Analysis in Portfolio Choice and Capital Markets*. Frank J. Fabozzi Associates, New Hope, PA, 2000.
- M. Stein, J. Branke, and H. Schmeck. Efficient implementation of an active set algorithm for large-scale portfolio selection. *Computers & Operations Research*, 35(12):3945–3961, 2008.
- M. Hirschberger, Y. Qi, and R. E. Steuer. Large-scale MV efficient frontier computation via a procedure of parametric quadratic programming. *European Journal of Operational Research*, 204(3):581–588, 2010.

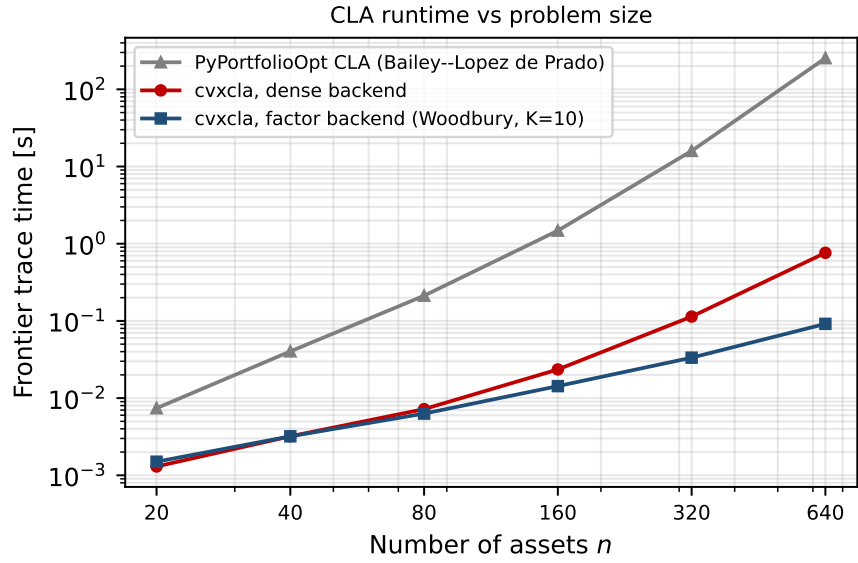


Figure 2: Frontier trace time as a function of the number of assets n (log-log). PyPortfolioOpt’s reference critical-line implementation (Bailey and López de Prado, 2013) scales like $\approx n^{3.4}$; `cvxcla`’s dense backend like $\approx n^{2.2}$; and its diagonal-plus-low-rank factor backend, solving the identical covariance through the Woodbury identity, like $\approx n^{1.3}$. At $n = 640$ the factor backend is roughly $2800\times$ faster than PyPortfolioOpt and $8\times$ faster than the dense backend.